

Surface-to-Air Missile Simulation Skeleton (A + B + JSON Factory)

This repository skeleton contains a minimal, modular C++ simulation framework with header/source pairs and a CMake build. It implements:

- Basic `State` container with arithmetic helpers.
- RK4 integrator (`Integrator`).
- Pluggable interfaces: `IAerodynamics`, `IGuidance`, `IAutopilot`.
- Simple implementations: `SimpleAero`, `PDAutopilot`, and **ProNav** guidance.
- **JSON factory** to wire modules and parameters from `configs/*.json` at runtime.
- `Missile` class that binds components and computes derivatives.
- `main.cpp` example that runs a single scenario and writes telemetry CSV **including target position**.
- `CMakeLists.txt` for building with Visual Studio Code (or any CMake environment).

Files (open each to view contents)

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(sam_sim_skeleton LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_POSITION_INDEPENDENT_CODE ON)

# Warnings
if(MSVC)
    add_compile_options(/W4 /permissive-)
else()
    add_compile_options(-Wall -Wextra -Wpedantic -Wconversion)
endif()

# Dependencies (header-only nlohmann_json)
include(FetchContent)
FetchContent_Declare(
    nlohmann_json
    URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz
)
FetchContent_MakeAvailable(nlohmann_json)

include_directories(${PROJECT_SOURCE_DIR}/include)
```

```

file(GLOB SRC
    src/*.cpp
    src/sim/*.cpp
    src/modules/*.cpp
)

add_executable(sam_sim ${SRC})

target_link_libraries(sam_sim PRIVATE nlohmann_json::nlohmann_json)

# Install target for convenience
install(TARGETS sam_sim RUNTIME DESTINATION bin)

```

include/sim/state.hpp

```

#ifndef SIM_STATE_HPP
#define SIM_STATE_HPP

#include <vector>
#include <cstdint>
#include <cassert>

namespace sim {

struct State {
    double t = 0.0;           // simulation time
    std::vector<double> x;     // generalized state vector

    State() = default;
    explicit State(size_t n): x(n, 0.0) {}

    size_t size() const { return x.size(); }
};

// Arithmetic helpers
State operator+(const State& a, const State& b);
State operator-(const State& a, const State& b);
State scaled(const State& a, double s);

} // namespace sim

#endif // SIM_STATE_HPP

```

src/sim/state.cpp

```
#include "sim/state.hpp"

namespace sim {

State operator+(const State& a, const State& b) {
    assert(a.size() == b.size());
    State r(a.size());
    r.t = a.t; // time usually handled separately
    for (size_t i = 0; i < a.size(); ++i) r.x[i] = a.x[i] + b.x[i];
    return r;
}

State operator-(const State& a, const State& b) {
    assert(a.size() == b.size());
    State r(a.size());
    r.t = a.t;
    for (size_t i = 0; i < a.size(); ++i) r.x[i] = a.x[i] - b.x[i];
    return r;
}

State scaled(const State& a, double s) {
    State r(a.size());
    r.t = a.t;
    for (size_t i = 0; i < a.size(); ++i) r.x[i] = a.x[i] * s;
    return r;
}

} // namespace sim
```

include/sim/integrator.hpp

```
#ifndef SIM_INTEGRATOR_HPP
#define SIM_INTEGRATOR_HPP

#include "sim/state.hpp"
#include <functional>

namespace sim {

using DerivFunc = std::function<State(double, const State&)>;

class RK4Integrator {
```

```

public:
    // single step RK4
    static State step(const DerivFunc& f, double t, const State& y, double h);
};

} // namespace sim

#endif // SIM_INTEGRATOR_HPP

```

src/sim/integrator.cpp

```

#include "sim/integrator.hpp"
#include "sim/state.hpp"

namespace sim {

State RK4Integrator::step(const DerivFunc& f, double t, const State& y, double
h) {
    State k1 = f(t, y);
    State y2 = y + scaled(k1, 0.5*h);
    State k2 = f(t + 0.5*h, y2);
    State y3 = y + scaled(k2, 0.5*h);
    State k3 = f(t + 0.5*h, y3);
    State y4 = y + scaled(k3, h);
    State k4 = f(t + h, y4);

    State out(y.size());
    out.t = y.t + h;
    for (size_t i = 0; i < y.size(); ++i) {
        out.x[i] = y.x[i] + (h/6.0) * (k1.x[i] + 2.0*k2.x[i] + 2.0*k3.x[i] +
k4.x[i]);
    }
    return out;
}

} // namespace sim

```

include/sim/components.hpp

```

#ifndef SIM_COMPONENTS_HPP
#define SIM_COMPONENTS_HPP

```

```

#include "sim/state.hpp"
#include <vector>
#include <array>
#include <memory>

namespace sim {

struct AeroForces {
    std::array<double,3> force{0.0,0.0,0.0};
    std::array<double,3> moment{0.0,0.0,0.0};
};

class IAerodynamics {
public:
    virtual ~IAerodynamics() = default;
    virtual AeroForces compute(const State& s, const std::vector<double>&
actuators) = 0;
};

class IGuidance {
public:
    virtual ~IGuidance() = default;
    // compute guidance command (interpreted by autopilot)
    virtual std::vector<double> guidance_command(const State& s, const State&
target) = 0;
};

class IAutopilot {
public:
    virtual ~IAutopilot() = default;
    // turn guidance command into actuator deflections / accel commands
    virtual std::vector<double> control(const State& s, const
std::vector<double>& guidanceCmd) = 0;
};

} // namespace sim

#endif // SIM_COMPONENTS_HPP

```

include/sim/missile.hpp

```

#ifndef SIM_MISSILE_HPP
#define SIM_MISSILE_HPP

#include "sim/state.hpp"

```

```

#include "sim/components.hpp"
#include <memory>

namespace sim {

class Missile {
public:
    Missile();

    // public state and modules
    State state; // layout: [px, py, pz, vx, vy, vz]

    std::unique_ptr<IAerodynamics> aero;
    std::unique_ptr<IGuidance> guidance;
    std::unique_ptr<IAutopilot> autopilot;

    // compute derivative given current state and actuator commands
    State derivative(double t, const State& s, const std::vector<double>&
actuatorCommands);
};

} // namespace sim

#endif // SIM_MISSILE_HPP

```

src/sim/missile.cpp

```

#include "sim/missile.hpp"
#include <cassert>

namespace sim {

Missile::Missile() {
    // default to 6-element state: pos(3), vel(3)
    state = State(6);
}

State Missile::derivative(double /*t*/, const State& s, const
std::vector<double>& actuatorCommands) {
    assert(s.size() == 6);
    State ds(6);
    ds.t = s.t;
    // positions' derivative = velocities
    ds.x[0] = s.x[3];
    ds.x[1] = s.x[4];

```

```

    ds.x[2] = s.x[5];

    // simple mass and force model: acceleration = F/m, here mass = 1 for
    simplicity
    AeroForces af = { {0,0,0}, {0,0,0} };
    if (aero) af = aero->compute(s, actuatorCommands);

    ds.x[3] = af.force[0];
    ds.x[4] = af.force[1];
    ds.x[5] = af.force[2];

    return ds;
}

} // namespace sim

```

include/sim/factory.hpp

```

#ifndef SIM_FACTORY_HPP
#define SIM_FACTORY_HPP

#include <string>
#include <memory>
#include <optional>
#include <vector>
#include "sim/missile.hpp"

namespace sim {

struct FactoryParams {
    // missile initial state
    State missile0;    // size 6
    // target initial state (size 6). Only px,py,pz,vx,vy,vz used.
    State target0;
    double t0 = 0.0;
    double tf = 60.0;
    double h = 0.01;
};

// Parse JSON at path and construct FactoryParams, plus configure missile
modules.
// Returns false and fills error on failure.
bool load_from_json(const std::string& path, Missile& missile, FactoryParams&
out, std::string& error);

```

```

} // namespace sim

#endif // SIM_FACTORY_HPP

```

src/sim/factory.cpp

```

#include "sim/factory.hpp"
#include "modules/simple_aero.hpp"
#include "modules/pd_autopilot.hpp"
#include "modules/ProNav.hpp"
#include <nlohmann/json.hpp>
#include <fstream>
#include <iostream>

namespace sim {

using nlohmann::json;

static bool read_json_file(const std::string& path, json& j, std::string& err) {
    std::ifstream f(path);
    if (!f) { err = "Cannot open config file: " + path; return false; }
    try { f >> j; }
    catch (const std::exception& e) { err = std::string("JSON parse error: ") +
e.what(); return false; }
    return true;
}

static double get_or(const json& j, const char* key, double def) {
    return j.contains(key) ? j.at(key).get<double>() : def;
}

bool load_from_json(const std::string& path, Missile& missile, FactoryParams&
out, std::string& error) {
    json j;
    if (!read_json_file(path, j, error)) return false;

    // timings
    out.t0 = j.value("t0", 0.0);
    out.tf = j.value("tf", 60.0);
    out.h = j.value("h", 0.01);

    // initial missile state
    out.missile0 = State(6);
    if (j.contains("initial_state")) {
        auto is = j["initial_state"];

```

```

    auto pos = is.value("pos", std::vector<double>{0,0,0});
    auto vel = is.value("vel", std::vector<double>{0,0,0});
    for (int i=0;i<3 && i<(int)pos.size();++i) out.missile0.x[i] = pos[i];
    for (int i=0;i<3 && i<(int)vel.size();++i) out.missile0.x[3+i] = vel[i];
}

// target state
out.target0 = State(6);
if (j.contains("target")) {
    auto tg = j["target"];
    auto pos = tg.value("pos", std::vector<double>{1000,0,100});
    auto vel = tg.value("vel", std::vector<double>{0,0,0});
    for (int i=0;i<3 && i<(int)pos.size();++i) out.target0.x[i] = pos[i];
    for (int i=0;i<3 && i<(int)vel.size();++i) out.target0.x[3+i] = vel[i];
}

// Modules
// Aero
if (j.contains("missile") && j["missile"].contains("aero")) {
    auto a = j["missile"]["aero"];
    std::string name = a.value("type", std::string("SimpleAero"));
    if (name == "SimpleAero") {
        double gain = get_or(a, "gain", 1.0);
        double damping = get_or(a, "damping", 0.0);
        double thrust = get_or(a, "thrust_accel", 0.0);
        // SimpleAero supports optional damping/thrust via parameters; pass
via ctor
        missile.aero = std::make_unique<modules::SimpleAero>(gain, damping,
thrust);
    } else {
        error = "Unknown aero type: " + name; return false;
    }
} else {
    missile.aero = std::make_unique<modules::SimpleAero>(1.0, 0.0, 0.0);
}

// Guidance
if (j.contains("missile") && j["missile"].contains("guidance")) {
    auto g = j["missile"]["guidance"];
    std::string name = g.value("type", std::string("ProNav"));
    if (name == "ProNav") {
        double N = get_or(g, "N", 3.5);
        double amax = get_or(g, "amax", 50.0);
        missile.guidance = std::make_unique<modules::ProNav>(N, amax);
    } else if (name == "PurePursuit") {
        double gain = get_or(g, "gain", 0.5);
        missile.guidance =
std::make_unique<modules::PurePursuitGuidance>(gain);
    }
}

```

```

    } else {
        error = "Unknown guidance type: " + name; return false;
    }
}

// Autopilot
if (j.contains("missile") && j["missile"].contains("autopilot")) {
    auto ap = j["missile"]["autopilot"];
    std::string name = ap.value("type", std::string("PD"));
    if (name == "PD") {
        double kp = get_or(ap, "kp", 1.0);
        double kd = get_or(ap, "kd", 0.0);
        missile.autopilot = std::make_unique<modules::PDAutopilot>(kp, kd);
    } else {
        error = "Unknown autopilot type: " + name; return false;
    }
}

return true;
}

} // namespace sim

```

include/modules/simple_aero.hpp

```

#ifndef MODULES_SIMPLE_AERO_HPP
#define MODULES_SIMPLE_AERO_HPP

#include "sim/components.hpp"

namespace modules {

class SimpleAero : public sim::IAerodynamics {
public:
    // gain scales commanded accel; optional linear damping and axial thrust
    accel
    SimpleAero(double accel_gain = 1.0, double damping = 0.0, double
    thrust_accel = 0.0);
    sim::AeroForces compute(const sim::State& s, const std::vector<double>&
    actuators) override;
private:
    double gain_;
    double damping_;
    double thrust_accel_;
};

```

```
} // namespace modules

#endif // MODULES_SIMPLE_AERO_HPP
```

src/modules/simple_aero.cpp

```
#include "modules/simple_aero.hpp"

namespace modules {

SimpleAero::SimpleAero(double accel_gain, double damping, double thrust_accel)
    : gain_(accel_gain), damping_(damping), thrust_accel_(thrust_accel) {}

// actuators interpreted as desired acceleration vector [ax, ay, az]
sim::AeroForces SimpleAero::compute(const sim::State& s, const
std::vector<double>& actuators) {
    sim::AeroForces out;
    if (actuators.size() >= 3) {
        out.force[0] = thrust_accel_ + gain_ * actuators[0] - damping_ * s.x[3];
        out.force[1] = gain_ * actuators[1] - damping_ * s.x[4];
        out.force[2] = gain_ * actuators[2] - damping_ * s.x[5];
    }
    return out;
}

} // namespace modules
```

include/modules/pure_pursuit_guidance.hpp

```
#ifndef MODULES_PURE_PURSUIT_GUIDANCE_HPP
#define MODULES_PURE_PURSUIT_GUIDANCE_HPP

#include "sim/components.hpp"

namespace modules {

class PurePursuitGuidance : public sim::IGuidance {
public:
    PurePursuitGuidance(double gain = 1.0);
    std::vector<double> guidance_command(const sim::State& s, const sim::State&
target) override;
};

}
```

```

private:
    double gain_;
};

} // namespace modules

#endif // MODULES_PURE_PURSUIT_GUIDANCE_HPP

```

src/modules/pure_pursuit_guidance.cpp

```

#include "modules/pure_pursuit_guidance.hpp"
#include <cmath>

namespace modules {

PurePursuitGuidance::PurePursuitGuidance(double gain): gain_(gain) {}

std::vector<double> PurePursuitGuidance::guidance_command(const sim::State& s,
const sim::State& target) {
    // simple vector from missile position to target position scaled by gain
    std::vector<double> cmd(3, 0.0);
    if (s.size() >= 3 && target.size() >= 3) {
        cmd[0] = gain_ * (target.x[0] - s.x[0]);
        cmd[1] = gain_ * (target.x[1] - s.x[1]);
        cmd[2] = gain_ * (target.x[2] - s.x[2]);
    }
    return cmd;
}

} // namespace modules

```

include/modules/pd_autopilot.hpp

```

#ifndef MODULES_PD_AUTOPILOT_HPP
#define MODULES_PD_AUTOPILOT_HPP

#include "sim/components.hpp"

namespace modules {

class PDAutopilot : public sim::IAutopilot {
public:

```

```

    PDAutopilot(double kp = 1.0, double kd = 0.1);
    std::vector<double> control(const sim::State& s, const std::vector<double>&
guidanceCmd) override;
private:
    double kp_;
    double kd_;
    std::vector<double> prev_cmd_ = {0.0,0.0,0.0};
};

} // namespace modules

#endif // MODULES_PD_AUTOPILOT_HPP

```

src/modules/pd_autopilot.cpp

```

#include "modules/pd_autopilot.hpp"

namespace modules {

PDAutopilot::PDAutopilot(double kp, double kd): kp_(kp), kd_(kd) {}

std::vector<double> PDAutopilot::control(const sim::State& /*s*/, const
std::vector<double>& guidanceCmd) {
    std::vector<double> out(3, 0.0);
    for (size_t i = 0; i < 3 && i < guidanceCmd.size(); ++i) {
        double err = guidanceCmd[i];
        double derr = guidanceCmd[i] - prev_cmd_[i];
        out[i] = kp_ * err + kd_ * derr;
        prev_cmd_[i] = guidanceCmd[i];
    }
    return out;
}

} // namespace modules

```

include/modules/ProNav.hpp

```

#ifndef MODULES_PRONAV_HPP
#define MODULES_PRONAV_HPP

#include "sim/components.hpp"

```

```

namespace modules {

class ProNav : public sim::IGuidance {
public:
    explicit ProNav(double N = 3.0, double amax = 100.0)
        : N_(N), amax_(amax) {}

    std::vector<double> guidance_command(const sim::State& m,
                                        const sim::State& t) override;

private:
    double N_;
    double amax_;
};

} // namespace modules

#endif // MODULES_PRONAV_HPP

```

src/modules/ProNav.cpp

```

#include "modules/ProNav.hpp"
#include <cmath>

namespace {
inline void cross(const double a[3], const double b[3], double c[3]) {
    c[0] = a[1]*b[2] - a[2]*b[1];
    c[1] = a[2]*b[0] - a[0]*b[2];
    c[2] = a[0]*b[1] - a[1]*b[0];
}
inline double dot(const double a[3], const double b[3]) {
    return a[0]*b[0] + a[1]*b[1] + a[2]*b[2];
}
inline double norm3(const double a[3]) {
    return std::sqrt(dot(a,a)) + 1e-9;
}
}

namespace modules {

std::vector<double> ProNav::guidance_command(const sim::State& m, const
sim::State& t) {
    double r[3]    = { t.x[0]-m.x[0], t.x[1]-m.x[1], t.x[2]-m.x[2] };
    double v_rel[3] = { t.x[3]-m.x[3], t.x[4]-m.x[4], t.x[5]-m.x[5] };
    const double R = norm3(r);

```

```

// unit LOS
double los_hat[3] = { r[0]/R, r[1]/R, r[2]/R };

// closing speed Vc = -dot(v_rel, los_hat)
const double Vc = - (v_rel[0]*los_hat[0] + v_rel[1]*los_hat[1] +
v_rel[2]*los_hat[2]);

// a_lat direction: los_hat x (v_rel x los_hat)
double v_cross_los[3]; cross(v_rel, los_hat, v_cross_los);
double dir[3];          cross(los_hat, v_cross_los, dir);

// scale
double scale = (N_ * Vc) / R;
double a_cmd[3] = { scale * dir[0], scale * dir[1], scale * dir[2] };

// saturate accel magnitude
double a_norm = std::sqrt(a_cmd[0]*a_cmd[0] + a_cmd[1]*a_cmd[1] +
a_cmd[2]*a_cmd[2]);
if (a_norm > amax_) {
    double s = amax_ / a_norm;
    a_cmd[0] *= s; a_cmd[1] *= s; a_cmd[2] *= s;
}

return { a_cmd[0], a_cmd[1], a_cmd[2] };
}

} // namespace modules

```

src/main.cpp

```

#include <iostream>
#include <fstream>
#include <memory>
#include <string>
#include "sim/state.hpp"
#include "sim/integrator.hpp"
#include "sim/missile.hpp"
#include "sim/factory.hpp"

using namespace sim;

int main(int argc, char** argv) {
    std::string config_path = (argc > 1) ? argv[1] : std::string("configs/
simple_scenario.json");

```

```

Missile missile;
FactoryParams params; std::string err;
if (!load_from_json(config_path, missile, params, err)) {
    std::cerr << "Config error: " << err << "
";
    return 1;
}

// Initialize states
missile.state = params.missile0; missile.state.t = params.t0;
State target = params.target0; // position + velocity

// simulation parameters
double t = params.t0;
double tf = params.tf;
double h = params.h;
double t_target = t; // time associated with 'target'

std::ofstream out("telemetry.csv");
out << "t,px,py,pz,vx,vy,vz,tx,ty,tz
";

State s = missile.state;
while (t < tf) {
    // --- Update target to current simulation time t (before guidance) ---
    double dt_target = t - t_target;
    if (dt_target != 0.0) {
        target.x[0] += target.x[3] * dt_target;
        target.x[1] += target.x[4] * dt_target;
        target.x[2] += target.x[5] * dt_target;
        t_target = t;
    }

    // --- Guidance and control (zero-order hold over this step) ---
    auto guidanceCmd = missile.guidance->guidance_command(s, target);
    auto control = missile.autopilot->control(s, guidanceCmd);

    // Derivative uses current control held constant during the integration
step
    DerivFunc f = [&](double tt, const State& yy) {
        (void)tt; // unused in this skeleton
        return missile.derivative(tt, yy, control);
    };

    State snw = RK4Integrator::step(f, t, s, h);
    snw.t = t + h;

    // --- Log telemetry (missile and target at step end) ---

```

```

        out << snw.t << ", "
        << snw.x[0] << ", " << snw.x[1] << ", " << snw.x[2] << ", "
        << snw.x[3] << ", " << snw.x[4] << ", " << snw.x[5] << ", "
        << target.x[0] << ", " << target.x[1] << ", " << target.x[2] << "
";

    s = snw;
    t += h;

    // Naive termination if near target (uses target at time t)
    double dx = s.x[0] - target.x[0];
    double dy = s.x[1] - target.x[1];
    double dz = s.x[2] - target.x[2];
    double dist2 = dx*dx + dy*dy + dz*dz;
    if (dist2 < 25.0) {
        std::cout << "Intercept (approx) at t=" << t << "
";
        break;
    }
}

out.close();
std::cout << "Simulation finished, telemetry.csv written." << std::endl;
return 0;
}

```

configs/simple_scenario.json

```

{
  "t0": 0.0,
  "tf": 60.0,
  "h": 0.01,
  "missile": {
    "aero": { "type": "SimpleAero", "gain": 1.0, "damping": 0.2,
    "thrust_accel": 0.0 },
    "guidance": { "type": "ProNav", "N": 3.5, "amax": 50.0 },
    "autopilot": { "type": "PD", "kp": 1.0, "kd": 0.0 }
  },
  "initial_state": {
    "pos": [0, 0, 0],
    "vel": [300, 0, 0]
  },
  "target": {
    "pos": [2000, 0, 1000],
    "vel": [150, 0, 0]
  }
}

```

```
}  
}
```

README.md

```
# SAM Simulation Skeleton (with JSON Factory)  
  
## Build  
```bash  
mkdir build && cd build
cmake ..
cmake --build . --config Release
```

## Run

```
./sam_sim # uses configs/simple_scenario.json
./sam_sim path/to/your_config.json
```

The program writes `telemetry.csv` with columns

```
t,px,py,pz,vx,vy,vz,tx,ty,tz
```

Use `plot_telemetry.py` to visualize.

## Config schema (summary)

- **t0, tf, h:** start time, end time, step size
- **initial\_state:** { pos:[x,y,z], vel:[vx,vy,vz] } for missile
- **target:** { pos:[x,y,z], vel:[vx,vy,vz] }
- **missile.aero:**
  - type: "SimpleAero"
  - parameters: gain, damping, thrust\_accel
- **missile.guidance:**
  - type: "ProNav" with N, amax
  - or type: "PurePursuit" with gain
- **missile.autopilot:**
  - type: "PD" with kp, kd ""

## Notes and next steps

- The factory wires modules and parameters from JSON.
- Next we can add: plugin loader (dlopen/LoadLibrary), Monte Carlo manager, and more physics fidelity.